

Guía de despliegue

Esta aplicación tiene un backend Node.js/Express y un frontend React (Vite). Puedes desplegarla de dos formas:

- A) Monolítica en cPanel (backend y frontend juntos)
- B) Separada: frontend en cPanel (<https://cotizador.aysafi.com>) y backend en un VPS (<https://emqx.aysafi.com>)

A continuación están ambos flujos. Si buscas el escenario "separado", ve directo a la sección B.

A) Quickstart cPanel (monolítico: backend + frontend)

En cPanel usaremos Application Manager (Node.js/Passenger) para correr el backend y servir la SPA.

1. Obtener el paquete listo (release)

- En tu entorno local puedes generar el paquete con `npm run package:cpanel`. Se crea una carpeta `release/cpanel-<timestamp>/` con un ZIP listo.
- Alternativamente, usa el ZIP ya generado que encuentres en `release/` (por ejemplo: `release/cpanel-20250924-001439.zip`).

1. Subir y descomprimir en cPanel

- Crea una carpeta en tu home (ej. `cotizador`).
- Sube el ZIP y descomprímelo ahí. Debes ver `backend/`, `frontend/dist/`, `.env.example`, `package.json`, etc.

1. Crear aplicación Node.js en cPanel

- Application root: la carpeta donde descomprimiste (ej. `/home/usuario/cotizador`).
- Application URL: el subdominio elegido (ej. `https://cotizador.tudominio.cl`).
- Startup file: `backend/server.js`.
- Versión de Node: 20.x.

1. Variables mínimas (en Application Manager)

- `JWT_SECRET`: secreto largo y único.
- `FRONTEND_URL`: ej. `https://cotizador.tudominio.cl`.
- (Opcional) `PUBLIC_API_BASE`: déjalo vacío si frontend y backend comparten dominio.
- (Opcional) `OUTPUT_DIR`: por defecto usa `backend/outputs` (junto a `app.js`).
- (SMTP si enviarás correos) `SMTP_HOST`, `SMTP_PORT`, `SMTP_USER`, `SMTP_PASS`, `SMTP_FROM`.

1. Instalar dependencias (Terminal de la App)

- Ejecuta en la raíz de la app: `npm ci`.
- Luego: `cd frontend && npm ci && cd ...`

1. Reiniciar y probar

- Reinicia la app desde Application Manager.
- Abre la URL pública y verifica:
 - La página carga sin errores.
 - Rutas profundas de la SPA como `/admin/login` funcionan.
 - `/outputs/*` sirve archivos si existen.
 - `/config.js` responde (runtime config) — opcional si usas config embebida.

1. Seguridad y notas

- Si sirves el frontend con Apache (docroot estático) en lugar de Passenger, usa el `.htaccess` de `frontend/.htaccess` incluido en el paquete para CSP y fallback SPA.
- En modo monolítico (Node/Passenger), los headers de `.htaccess` no aplican.

1. Permisos

- Asegura que `backend/outputs/` sea escribible por la app.
- No subas `.env` al repo; usa variables de entorno en cPanel.
- Rota `JWT_SECRET` y credenciales SMTP tras pruebas.

Alternativa (si no usas el ZIP de release):

- Sube el repo (sin `node_modules/`), construye el frontend con `npm run frontend:build` y continúa desde el paso 3.

SMTP troubleshooting (Nodemailer)

- `ETIMEDOUT/Greeting never received`: revisa host/puerto/seguridad; prueba `npm run verify:smtp` y `npm run send:test-email`.

B) Separado: frontend en cPanel y backend en VPS

Objetivo: servir el frontend estático en `https://cotizador.aysafi.com` (cPanel) y el backend en `https://emqx.aysafi.com` (VPS). El frontend llamará al backend vía `PUBLIC_API_BASE` usando `window.__APP_CONFIG__` cargado desde `config.js`.

1) Backend en VPS (<https://emqx.aysafi.com>)

Requisitos en el VPS (Ubuntu/Debian típico):

- DNS del dominio `emqx.aysafi.com` apuntando a la IP del VPS
- Node.js 20 y npm
- Nginx como reverse proxy con SSL (Let's Encrypt)
- Acceso SSH y permisos para crear un servicio (systemd)

Pasos:

1. Clonar e instalar dependencias

```
sudo mkdir -p /opt/cotizador && sudo chown $USER:$USER /opt/cotizador
cd /opt/cotizador
git clone https://github.com/asdrubalfuentes/cotizador .
npm ci
```

1. Variables de entorno del backend (archivo `.env` en `/opt/cotizador`)

```
# URL públicas
FRONTEND_URL=https://cotizador.aysafi.com
PUBLIC_API_BASE=https://emqx.aysafi.com

# Seguridad
JWT_SECRET=pon-aqui-un-secreto-largo-unico
ADMIN_PASSWORD=elige-una-clave-admin

# SMTP (si enviarás correos)
SMTP_HOST=smtp.tu-proveedor.com
SMTP_PORT=587
SMTP_USER=usuario@tu-dominio.com
SMTP_PASS=tu-pass

# Otros (opcionales)
OUTPUT_DIR=/var/lib/cotizador/outputs
MORGAN_FORMAT=combined
```

1. Directorio de salidas (si usas ruta externa)

```
sudo mkdir -p /var/lib/cotizador/outputs
sudo chown -R $USER:$USER /var/lib/cotizador
```

1. Servicio systemd (opcional recomendado)

Archivo: `/etc/systemd/system/cotizador.service`

```
[Unit]
Description=Cotizador Backend
After=network.target

[Service]
Type=simple
WorkingDirectory=/opt/cotizador
Environment=NODE_ENV=production
ExecStart=/usr/bin/node backend/server.js
Restart=always
RestartSec=5
```

```
# User=cotizador ; si creas un usuario de servicio
```

[Install]

```
WantedBy=multi-user.target
```

Activar e iniciar:

```
sudo systemctl daemon-reload  
sudo systemctl enable cotizador --now
```

1. Nginx reverse proxy con SSL y SSE

Archivo: `/etc/nginx/sites-available/cotizador-backend`

```
server {  
    listen 80;  
    server_name emqx.aysafi.com;  
    location /.well-known/acme-challenge/ { root /var/www/html; }  
    location / { return 301 https://$host$request_uri; }  
}  
  
server {  
    listen 443 ssl http2;  
    server_name emqx.aysafi.com;  
  
    ssl_certificate /etc/letsencrypt/live/emqx.aysafi.com/fullchain.pem;  
    ssl_certificate_key /etc/letsencrypt/live/emqx.aysafi.com/privkey.pem;  
  
    client_max_body_size 10m; # subir logos (la app limita 5MB)  
  
    location / {  
        proxy_pass http://127.0.0.1:3000; # puerto donde corre Node  
        proxy_http_version 1.1;  
        proxy_set_header Host $host;  
        proxy_set_header X-Real-IP $remote_addr;  
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;  
        proxy_set_header X-Forwarded-Proto $scheme;  
  
        # SSE (EventSource): sin buffer y con timeouts amplios  
        proxy_read_timeout 3600s;  
        proxy_send_timeout 3600s;  
        proxy_buffering off;  
    }  
}
```

Activar sitio y recargar:

```
sudo ln -s /etc/nginx/sites-available/cotizador-backend /etc/nginx/sites-enabled/  
sudo nginx -t && sudo systemctl reload nginx
```

1. Comprobaciones rápidas del backend

- `curl https://emqx.aysafi.com/` → JSON { ok: true, ... }
- `curl -I https://emqx.aysafi.com/api/events` → Content-Type: text/event-stream

Notas CORS/SSE:

- CORS: el backend usa `cors()` abierto; para restringir: `cors({ origin: 'https://cotizador.aysafi.com' })`.
- SSE: con `proxy_buffering off` y timeouts altos, EventSource funciona estable tras proxies.

Tipos de cambio (rates):

- El frontend consulta tasas vía el backend (`/api/rates`) para evitar CORS y mejorar confiabilidad.
- En caso de fallo de la fuente externa, el backend responde 200 con los últimos valores en caché (si existen) o ceros y `source: 'unavailable'`.
- Variables de entorno clave: `RATES_SOURCE=backend|disabled`, `QUOTE_SKIP_RATES`, `PDF_SKIP_RATES`, `RATES_TIMEOUT_MS`.

HTTPS directo en Node (alternativa)

Si no deseas usar Nginx delante, el backend puede exponer HTTPS directamente (útil para pruebas o despliegues simples). Ya viene soportado en `backend/server.js`.

Requisitos y pasos similares a la guía principal.